

Contents

1	Predicción estructural clásica	1
1.1	El problema de la predicción estructural antes del Deep Learning	1
1.2	Modelado por homología (<i>Comparative Modeling</i>)	1
1.2.1	El principio evolutivo de conservación estructural	2
1.2.2	Flujo de trabajo (<i>Pipeline</i>) de modelado comparativo	2
1.2.3	El estándar MODELLER: satisfacción de restricciones espaciales	3
1.3	Reconocimiento de plegamiento (<i>Threading / Fold Recognition</i>)	3
1.4	Predicción <i>De Novo</i> y <i>Ab Initio</i> : la suite Rosetta	3
1.4.1	Ensamblaje estadístico de fragmentos	4
1.4.2	Muestreo conformacional: Monte Carlo con criterio de Metropolis	4
1.4.3	Funciones de energía (Rosetta REF2015)	4
1.4.4	Validación defensiva del parámetro de temperatura	4
1.5	Métricas de evaluación y calidad de modelos predichos	5
1.5.1	Métrica global independiente de longitud: TM-score	5
1.5.2	Validación estereoquímica: MolProbity Clashscore	5
1.5.3	Estimación de calidad absoluta: QMEAN	5
1.6	Diseño computacional en Python	5
1.7	Peligros computacionales en predicción estructural	6
1.8	Actividad de depuración: 3 errores en flujo de predicción	6
1.9	Síntesis operativa y tablas de diagnóstico	6
1.9.1	Tabla de selección de metodología de predicción clásica	6
1.9.2	Tabla comparativa de métricas de calidad estructural	6
1.10	Glosario conceptual	7
1.11	Contexto histórico y transición hacia BC-CH13	7
1.12	Fuentes y reproducibilidad	7
	Anexo práctico · Métricas de predicción estructural, criterio de Metropolis y evaluación de choques	9

Chapter 1

Predicción estructural clásica

1.1 El problema de la predicción estructural antes del Deep Learning

En el capítulo anterior (BC-CH11) analizamos la evidencia física experimental que sustenta las estructuras tridimensionales depositadas en el *Protein Data Bank* (PDB). Sin embargo, existe una asimetría colosal entre el número de secuencias genómicas conocidas (> 250 millones en UniProtKB) y el número de estructuras resueltas experimentalmente por cristalografía, RMN o Cryo-EM ($< 250\,000$ entradas). La determinación experimental es lenta, costosa y a menudo inviable para proteínas insolubles o complejas.

Esencial

Ante este reto, la biología computacional formuló una de sus preguntas más ambiciosas: **¿es posible predecir la estructura tridimensional de una proteína a partir únicamente de su secuencia de aminoácidos?**

Antes de la irrupción de las redes neuronales profundas contemporáneas como AlphaFold (objeto de estudio en BC-CH13), la comunidad científica desarrolló durante más de cuatro décadas tres grandes paradigmas analíticos y termodinámicos:

1. **Modelado por homología (*Comparative Modeling*):** cuando existen estructuras experimentales con alta similitud de secuencia ($> 30\%$).
2. **Reconocimiento de plegamiento (*Threading / Fold Recognition*):** cuando la similitud de secuencia cae en la zona de penumbra ($< 30\%$), pero la proteína puede adoptar uno de los ≈ 1400 plegamientos canónicos conocidos.
3. **Predicción *De Novo / Ab Initio* (Rosetta):** cuando no existe ninguna plantilla conocida, ensamblando fragmentos estadísticos mediante muestreo estocástico de Monte Carlo.

La ruta **esencial** cubre los principios del modelado por homología y MODELLER, el reconocimiento de plegamiento (*threading*), el muestreo de Monte Carlo con criterio de Metropolis en Rosetta, la métrica TM-score y la validación de choques con MolProbity Clashscore; 120–140 minutos. La ruta de **estudio** profundiza en las funciones de energía REF2015, actividad de depuración y el anexo práctico; 60–80 minutos adicionales.

Esencial

La predicción estructural clásica transforma el problema biológico del plegamiento en un problema de optimización matemática sobre el paisaje de energía conformacional.

1.2 Modelado por homología (*Comparative Modeling*)

1.2.1 El principio evolutivo de conservación estructural

El modelado por homología se sustenta en el principio descubierto por Chothia y Lesk (1986): **la estructura tridimensional está mucho más conservada en la evolución que la secuencia primaria de aminoácidos**. Durante la divergencia de dos proteínas homólogas, la secuencia acumula mutaciones neutras mientras el esqueleto tridimensional del núcleo hidrofóbico se preserva para mantener la función biológica.

- **Zona segura (> 30% de identidad):** El modelo predicho alcanza habitualmente una precisión atómica comparable a una estructura experimental de resolución media (RMSD $\approx$ 1–2 Å).
- **Zona de penumbra (*Twilight Zone*, 20–30% de identidad):** El alineamiento de secuencia se vuelve ambiguo; el modelado por homología estándar fracasa y requiere métodos de *threading*.
- **Zona de medianoche (< 20% de identidad):** El ruido estadístico impide detectar homología mediante alineamiento simple.

El modelado por homología aprovecha la conservación estructural evolutiva cuando la identidad de secuencia supera el umbral seguro del 30%.

Evaluación computacional de identidad de secuencia: Dadas dos secuencias modelo alineadas de 20 aminoácidos:

Seq1: ACDEFGHIKLMNPQRSTVWY

Seq2: ACDEFGHIKLMNPQRSTVWA

Con 19 coincidencias exactas sobre 20 posiciones:

$$\%ID = \frac{19}{20} \times 100 = \mathbf{95.00\%}$$

`calculate_sequence_identity` calcula el porcentaje de identidad evaluando el par modelo de 20 residuos exactamente al 95.00%. Se reproduce con `verification/chapter_checks.py identity`.

1.2.2 Flujo de trabajo (*Pipeline*) de modelado comparativo

El proceso de modelado por homología se estructura en cuatro etapas secuenciales:

1. **Búsqueda de plantillas (*Template Search*):** Se compara la secuencia diana (*query*) contra la base de datos PDB mediante herramientas de alta sensibilidad: BLASTp, PSI-BLAST (matrices PSSM iterativas) o perfiles de modelos ocultos de Markov (HMM-HMM con HHpred).
2. **Alineamiento diana-plantilla (*Target-Template Alignment*):** Es la fase más crítica. Un único desfase de alineamiento (*alignment shift*) de una posición rota toda una hélice α 100° alrededor de su eje cilíndrico o invierte la dirección de los residuos en una hebra β , destruyendo el modelo resultante.
3. **Construcción del modelo (*Model Building*):**
 - Se transfieren las coordenadas atómicas de las regiones estructuralmente conservadas (SCRs, *Structurally Conserved Regions*).

- Se modelan las inserciones y deleciones (*loops* o bucles variables) mediante búsqueda en bibliotecas de fragmentos o relajación de cadenas.
 - Se construyen las cadenas laterales optimizando la conformación de rotámeros mediante librerías dependientes del esqueleto.
4. **Evaluación y refinamiento:** Minimización de energía para eliminar choques estéricos y evaluación de calidad estereoquímica.

El flujo de modelado comparativo encadena la búsqueda de plantillas (PSI-BLAST, HHpred), el alineamiento diana-plantilla, la construcción del modelo mediante restricciones espaciales en MODELLER y el refinamiento de bucles.

El desfase de alineamiento (*alignment shift*) representa el error más grave en modelado por homología al introducir rotaciones y deformaciones estructurales globales en el esqueleto polipeptídico.

1.2.3 El estándar MODELLER: satisfacción de restricciones espaciales

El programa **MODELLER** (Sali y Blundell, 1993) revolucionó el modelado comparativo al abandonar la simple copia rígida de fragmentos. En su lugar, extrae **restricciones espaciales** de las plantillas (distancias interatómicas $C\alpha-C\alpha$, ángulos diedros ϕ, ψ, χ) y las modela como funciones de densidad de probabilidad (PDF) gaussianas continuas. El modelo tridimensional final se obtiene optimizando conjuntamente la función objetivo global mediante gradiente conjugado y dinámica molecular simulada.

1.3 Reconocimiento de plegamiento (*Threading / Fold Recognition*)

Cuando la identidad de secuencia cae en la zona de penumbra ($< 30\%$), los alineamientos primarios de secuencia no permiten detectar parentesco evolutivo. Sin embargo, dado que el repertorio total de plegamientos proteicos en la naturaleza es limitado ($\approx 1300-2000$ familias según CATH/SCOP), existe una alta probabilidad de que la proteína diana comparta la arquitectura de una estructura ya resuelta.

El método de **enhebrado** (*Threading*) analiza físicamente si la secuencia problema es energéticamente compatible con cada uno de los moldes estructurales de la biblioteca PDB:

1. Se monta la secuencia diana sobre la geometría 3D de la plantilla.
2. Se calcula la energía del ensamblaje utilizando potenciales estadísticos dependientes de la distancia y potenciales de solvatación e hidrofobicidad ambiental.
3. Software representativo: **I-TASSER** (Yang y Zhang, 2015) y **HHpred** (Söding, 2005).

El reconocimiento de plegamiento (*threading*) mapea secuencias en la zona de penumbra ($< 30\%$ de identidad) sobre moldes estructurales canónicos del PDB mediante potenciales estadísticos de energía (I-TASSER).

1.4 Predicción *De Novo* y *Ab Initio*: la suite Rosetta

Cuando no existe ninguna plantilla conocida ni homóloga en el PDB, se recurre a la predicción **Ab Initio** o *De Novo*. El enfoque más exitoso fue desarrollado por David Baker y colaboradores en la suite **Rosetta** (Rohl et al., 2004).

1.4.1 Ensamblaje estadístico de fragmentos

Rosetta combina dos principios fundamentales:

- **Estructura local dictada por fragmentos:** Ventanas cortas de la secuencia (3 y 9 aminoácidos) adoptan conformaciones locales preferidas que se extraen estadísticamente del PDB.
- **Ensamblaje global mediante campos de fuerza:** La estructura terciaria global se forma ensamblando estos fragmentos mediante una búsqueda estocástica guiada por una función de energía fisicoquímica.

La predicción *de novo* y *ab initio* en Rosetta combina el ensamblaje de fragmentos estructurales cortos (3–9 residuos) con muestreo conformacional de Monte Carlo y criterio de Metropolis.

1.4.2 Muestreo conformacional: Monte Carlo con criterio de Metropolis

El algoritmo de Monte Carlo explora el espacio conformacional en coordenadas internas (modificando los ángulos de torsión ϕ, ψ mientras mantiene fijos los enlaces covalentes). En cada paso:

1. Se aplica una perturbación conformacional (sustitución de un fragmento o cizalla de torsión), produciendo una variación de energía $\Delta E = E_{\text{nuevo}} - E_{\text{actual}}$.
2. Se evalúa el **criterio de Metropolis**:

$$P(\text{aceptación}) = \begin{cases} 1.0 & \text{si } \Delta E \leq 0 \\ e^{-\Delta E/T} & \text{si } \Delta E > 0 \end{cases}$$

Esta regla permite aceptar temporalmente conformaciones desfavorables ($\Delta E > 0$) con probabilidad finita, evitando que el algoritmo quede atrapado en mínimos locales del paisaje energético.

Evaluación computacional del criterio de Metropolis: Para $\Delta E = 300.0$ y temperatura efectiva $T = 300.0$:

$$P = e^{-300.0/300.0} = e^{-1} \approx \mathbf{0.367879}$$

El criterio de Metropolis acepta cambios conformacionales con $P = \exp(-\Delta E/T)$ cuando $\Delta E > 0$, evaluando $\Delta E = 300.0$ a $T = 300.0$ exactamente a 0.367879. Se reproduce con `verification/chapter_checks.py metropolis -delta_e 300.0 -temp 300.0`.

1.4.3 Funciones de energía (Rosetta REF2015)

Las funciones de energía de Rosetta (REF2015) integran potenciales de Lennard-Jones con endurecimiento progresivo (*ramping*), solvatación implícita de Lazaridis-Karplus y enlaces de hidrógeno orientados.

1.4.4 Validación defensiva del parámetro de temperatura

`metropolis_acceptance_probability` lanza `ValueError` cuando la temperatura es menor o igual a cero ($T \leq 0$). Se reproduce con `verification/practice_checks.py domain_guard -temp 0.0`.

1.5 Métricas de evaluación y calidad de modelos predichos

1.5.1 Métrica global independiente de longitud: TM-score

A diferencia del RMSD tradicional (BC-CH11), que es extremadamente sensible a desplazamientos de un único bucle flexible y depende del tamaño de la proteína, el **TM-score** (Zhang y Skolnick, 2004) normaliza las distancias mediante una escala dependiente de la longitud:

$$d_0(L) = 1.24 \sqrt[3]{L_{\text{target}} - 15} - 1.8, \quad \text{TM-score} = \frac{1}{L_{\text{target}}} \sum_{i=1}^N \frac{1}{1 + \left(\frac{d_i}{d_0}\right)^2}$$

- Rango: $[0, 1]$. Para estructuras idénticas evalúa exactamente a **1.000000**.
- TM-score > 0.5 : Confirma con significación estadística que el modelo y la referencia comparten el mismo plegamiento global.
- TM-score < 0.17 : Similitud correspondiente a dos proteínas no relacionadas al azar.

El TM-score cuantifica la similitud de plegamiento de forma independiente de la longitud (TM > 0.5 confirma el mismo plegamiento), evaluando estructuras idénticas exactamente a 1.000000. Se reproduce con `verification/chapter_checks.py tm_score`.

1.5.2 Validación estereoquímica: MolProbity Clashscore

El **Clashscore** (Chen et al., 2010; Richardson lab) evalúa la presencia de choques atómicos estéricos desfavorables (solapamiento $> 0.4 \text{ \AA}$):

$$\text{Clashscore} = \frac{N_{\text{choques}}}{N_{\text{átomos}}} \times 1000$$

Evaluación computacional: Para 2 choques detectados en un modelo de 1000 átomos:

$$\text{Clashscore} = \frac{2}{1000} \times 1000 = \mathbf{2.000000}$$

Un modelo de alta calidad presenta habitualmente Clashscore < 5.0 .

El Clashscore de MolProbity mide el número de superposiciones estéricas graves por cada 1000 átomos, evaluando 2 choques en 1000 átomos a 2.000000. Se reproduce con `verification/chapter_checks.py clashscore -clashes 2 -atoms 1000`.

1.5.3 Estimación de calidad absoluta: QMEAN

La estimación de calidad absoluta con QMEAN / QMEAN Z-score valida el empaquetamiento y la solvatación energética sin requerir una estructura experimental de referencia.

1.6 Diseño computacional en Python

Presentamos la implementación de las funciones de evaluación:

Listing 1.1: Calculo de identidad y aceptacion de Metropolis en Python.

```
import math

def calculate_sequence_identity(seq1: str, seq2: str) -> float:
    if len(seq1) != len(seq2) or len(seq1) == 0:
```

```

        raise ValueError("Longitudes invalidas")
    matches = sum(1 for a, b in zip(seq1, seq2) if a == b and a != "-")
    return (matches / len(seq1)) * 100.0

def metropolis_acceptance(delta_e: float, temp: float = 300.0) -> float:
    if temp <= 0:
        raise ValueError("Temperatura debe ser positiva")
    if delta_e <= 0:
        return 1.0
    return math.exp(-delta_e / temp)

```

1.7 Peligros computacionales en predicción estructural

1. **Modelado por homología en la zona de penumbra sin *threading*:** Intentar forzar un modelo comparativo cuando la identidad es $< 25\%$ produce alineamientos erróneos y modelos deformados.
2. **Ignorar choques estéricos y residuos prohibidos en Ramachandran:** Utilizar modelos predichos con Clashscore > 20 o $< 90\%$ de residuos en regiones permitidas conduce a simulaciones de dinámica molecular inestables o cribados virtuales erróneos.

El uso de modelado por homología sin plantillas con identidad $> 25\%$ y la omisión de comprobaciones de choques atómicos y Ramachandran constituyen errores críticos de modelado.

1.8 Actividad de depuración: 3 errores en flujo de predicción

Localice los tres errores en la siguiente función de simulación:

Listing 1.2: Funcion con 3 errores de logica en evaluacion de prediccion.

```

def simulate_monte_carlo_step(delta_e, temp):
    # Error 1: no valida temperatura positiva produciendo division por cero
    # Error 2: invierte el signo de delta_e aceptando conformaciones de alta energia
    # Error 3: compara identidad sin normalizar por la longitud de alineamiento
    prob = math.exp(delta_e / temp)
    return prob

```

1.9 Síntesis operativa y tablas de diagnóstico

1.9.1 Tabla de selección de metodología de predicción clásica

Rango de Identidad	Metodología Óptima	Software Canónico	Precisión Típica
$> 30\%$	Modelado por Homología	MODELLER, SWISS-MODEL	Alta (RMSD $\approx 1-2$ Å)
$20-30\%$	Reconocimiento Threading	/ I-TASSER, HHpred	Media (TM-score > 0.5)
$< 20\%$	Predicción <i>Ab Initio</i>	Rosetta	Variable (2-6 Å)

1.9.2 Tabla comparativa de métricas de calidad estructural

Métrica	Rango	Requiere Referencia	Enfoque
RMSD	$0 \rightarrow \infty$ Å	Sí	Global (sensible a bucles)
TM-score	0.0–1.0	Sí	Global (independiente de tamaño)
MolProbity Clash	$0 \rightarrow \infty$	No (intrínseca)	Estereoquímica atómica
QMEAN Z-score	Desv. estándar	No (intrínseca)	Empaquetamiento y solvatación

1.10 Glosario conceptual

- **Homología comparativa:** modelado basado en plantillas homólogas.
- **Twilight Zone:** zona de penumbra (20–30% de identidad).
- **Alignment Shift:** desfase de alineamiento diana-plantilla.
- **Threading:** enhebrado de secuencias sobre moldes 3D.
- **MODELLER:** modelado por restricciones espaciales gaussianas.
- **Rosetta:** suite de predicción *ab initio* por fragmentos.
- **Metropolis:** criterio de aceptación estocástico en Monte Carlo.
- **TM-score:** métrica de superposición invariante a longitud.
- **Clashscore:** recuento de choques estéricos en MolProbity.
- **QMEAN:** estimación de calidad absoluta de modelos.

1.11 Contexto histórico y transición hacia BC-CH13

Este capítulo cubre los paradigmas clásicos de predicción analítica y heurística que establecieron los cimientos de la biología estructural computacional contemporánea.

La comprensión de las limitaciones del muestreo estocástico y la dependencia de plantillas homólogas da paso en BC-CH13 a las redes neuronales de atención geométrica y coevolución (AlphaFold).

Los métodos clásicos aquí analizados demostraron que el plegamiento proteico podía formularse como un problema de optimización computacional. Sin embargo, el muestreo estocástico de paisajes de energía rugosos y la búsqueda de plantillas encontraban barreras formidables ante proteínas grandes o sin homólogos cercanos. Esta frontera fue radicalmente superada con la llegada del aprendizaje profundo, los alineamientos múltiples evolutivos (MSA) y la arquitectura Evoformer en **AlphaFold**, tema central del siguiente capítulo (**BC-CH13: Predicción estructural con IA y evaluación de confianza**).

1.12 Fuentes y reproducibilidad

Este capítulo se fundamenta en las diapositivas docentes (20251127_estprot_2.pptx, Álvaro Serrano Navarro), en Sali y Blundell (1993), Rohl et al. (2004), Zhang y Skolnick (2004) y Chen et al. (2010). Todos los cálculos se reproducen mediante `verification/chapter_checks.py`.

Anexo práctico · Métricas de predicción estructural, criterio de Metropolis y evaluación de choques

Pregunta, producto y separación de materiales

¿Puede implementar un módulo en Python que evalúe la identidad de secuencia en zonas de homología, simule el criterio estocástico de aceptación de Metropolis en algoritmos de Monte Carlo (Rosetta), calcule la métrica global TM-score de superposición estructural y cuantifique la densidad de choques atómicos Clashscore de MolProbity de forma completamente determinista? Este laboratorio responde afirmativamente implementando cada componente sin dependencias opacas.

El producto individual contiene: el evaluador de identidad de secuencia para modelado por homología, el simulador del criterio de Metropolis, el cálculo analítico de TM-score, el evaluador de choques estéricos Clashscore, la perturbación de energía, el control de excepciones de temperatura y la defensa conceptual escrita.

El anexo es autónomo e independiente: no requiere paquetes externos. Emplea `verification/practice_checker` como oráculo determinista.

Mapa de ruta, tiempos y dedicación

Bloque de trabajo	Minutos	Producto entregable
1 · Entorno y diagnóstico	5	Comprobación del intérprete Python 3.9
2 · Cálculo ancla (Identidad, Metropolis, TM-score, Clash)	20–25	%ID = 95.0%, $P = 0.367879$, TM = 1.0, Clash = 2.0
3 · Perturbación energética de Metropolis	15–20	Evaluación de $\Delta E = 600.0 \implies P = 0.135335$
4 · Guarda de dominio	10–15	Captura de <code>ValueError</code> ante temperatura $T \leq 0$
5 · Preguntas de control de predicción	5–10	Preguntas sobre MODELLER, Threading y Rosetta
6 · Verificación y empaquetado	5	Comprobación final y empaquetado

La ruta de este anexo suma 75–100 minutos y produce evidencia comprobable y verificable en cada bloque de trabajo sin paquetes externos.

1 · Entorno y diagnóstico

Verifique que su entorno dispone de Python 3.9 o superior:

Listing 1.3: Comprobacion de interprete y oraculo.

```
python3 --version
python3 verification/practice_checks.py --help
```

2 · Cálculo ancla: identidad, Metropolis, TM-score y Clashscore

Dadas las condiciones estándar de ensayo:

1. Secuencias de prueba de 20 residuos (ACDEFGHIKLMNPQRSTVWY y ACDEFGHIKLMNPQRSTVWA): evalúa a identidad del **95.00%**.
2. Variación de energía $\Delta E = 300.0$ a temperatura $T = 300.0$: la probabilidad de aceptación de Metropolis evalúa a **0.367879** (e^{-1}).
3. Superposición de 4 átomos idénticos ($L = 4, d_i = 0$): el TM-score evalúa a **1.000000**.
4. Conteo de 2 choques en 1000 átomos: el Clashscore evalúa a **2.000000**.

Listing 1.4: Ejecucion del calculo ancla.

```
python3 verification/practice_checks.py anchor
```

El modelo ancla valida la identidad de secuencia al 95.00%, la probabilidad de Metropolis de 0.367879, el TM-score perfecto de 1.000000 y el Clashscore de 2.000000. Se reproduce con `verification/practice_checks.py anchor`.

3 · Perturbación: incremento de barrera energética en Monte Carlo

Evalúe la probabilidad de aceptación cuando la variación desfavorable de energía aumenta a $\Delta E = 600.0$ a la misma temperatura $T = 300.0$:

Listing 1.5: Calculo de probabilidad de Metropolis perturbada.

```
python3 verification/practice_checks.py perturbation
```

La perturbacion de energia con $\Delta E = 600.0$ a $T = 300.0$ evalua la probabilidad de aceptacion de Metropolis exactamente a 0.135335. Se reproduce con `verification/practice_checks.py perturbation`.

4 · Guarda de dominio y control de excepciones

Compruebe que el simulador rechaza temperaturas físicamente imposibles ($T \leq 0$):

Listing 1.6: Guarda de dominio ante temperatura no positiva.

```
python3 verification/practice_checks.py domain_guard --temp 0.0
```

La guarda de dominio rechaza temperaturas menores o iguales a cero lanzando un `ValueError` que previene divisiones por cero. Se reproduce con `verification/practice_checks.py domain_guard -temp 0.0`.

5 · Verificación y entrega

La verificación de entrega confirma la ejecución autónoma bajo el intérprete estándar de Python 3.9 sin librerías externas opacas.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%):** Ejecución correcta y reproducible de los scripts de comprobación (`chapter_checks.py` y `practice_checks.py`).
- **2.0 puntos (20%):** Implementación de las funciones de identidad de secuencia y criterio de Metropolis (Bloque 2).
- **2.0 puntos (20%):** Cálculo de TM-score analítico, Clashscore y control de excepciones en la guarda (Bloques 3 y 4).
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre las diferencias entre modelado por homología y *threading* y el impacto de los desfases de alineamiento.

Lista de autocorrección

- Verifiqué que mi versión de Python es ≥ 3.9 .
- La identidad de secuencia evaluó al 95.00%.
- Metropolis ancla resultó 0.367879.
- El TM-score idéntico resultó 1.000000.
- El Clashscore evaluó a 2.000000.
- La perturbación evaluó a 0.135335.
- Verifiqué que $T = 0.0$ lanza `ValueError`.
- Redacté la defensa conceptual individual.

Defensa conceptual

En 100–150 palabras, justifique por qué un desfase de una sola posición en el alineamiento diana-plantilla (*alignment shift*) resulta fatal para el modelado por homología de una hélice α , y explique cómo el reconocimiento de plegamiento (*threading*) permite identificar modelos estructurales cuando la identidad de secuencia cae por debajo del 25%.

Fuentes y reproducibilidad

Este anexo se fundamenta en las diapositivas de predicción de estructura (20251127_estprot_2.pptx), en Sali y Blundell (1993), Rohl et al. (2004), Zhang y Skolnick (2004) y Chen et al. (2010), y se valida al 100% mediante `verification/practice_checks.py`.